ECSE 429 – Part A Written Report

Rest api todo list manager 1.5.5 · Team: Kevin He (261167637), Nathan Audegond (261171251), Eric Deng (261138815) · Repo: https://github.com/kvinhe/ECSE429_PartA

1. Deliverables

- **Session notes** (PDF, 4 sessions × 45 min): todos, projects, categories, plus an extra interoperability session – `session_notes/CH1-CH4_*_Notes.pdf`.
- **Exploratory scripts** (curl/bash) and their logs – `exploratory/`, `logs/exploratory_*.log`.
- **Unit test suite** (Python 3 + pytest + requests, hand-written, no generated tests) – `tests/`, 185 tests in 12 modules.
- **Bug reports** (12 bugs, form below) – `bug_reports.md`.
- **Run logs** for default order, three seeded random orders, and a service-down run – `logs/run_*.log`.
- **Video** of the suite: default order, two seeded random orders, service stopped – `PartA_UnitTests_Video.mp4`.

2. Findings of exploratory testing

The API exposes three entities – **todos**, **projects**, **categories** – each with collection (`/x`) and instance (`/x/:id`) endpoints, plus relationship endpoints (todo: `tasksof`, `categories`; project: `tasks`, `categories`; category: `todos`, `projects`). JSON is the default; XML is selected with Content-Type / Accept. The swagger file describes 127 operations on 19 paths; most non-CRUD verbs are documented as 405.

Capability	Status
CRUD on todos / projects / categories; POST amend, PUT replace; strict validation with `errorMessages`	Documented
Link/unlink relationships (POST body `{"id":..}`, DELETE instance); 405 on unsupported verbs	Documented
XML request and response bodies	Documented (/docs)
Filtering by field (`?title=`, `?doneStatus=`); unknown filter fields silently ignored	Undocumented
Create a todo directly under a project (`POST /projects/:id/tasks {"title":..}`)	Undocumented
HEAD / OPTIONS (Allow header); HTML pages `/gui`, `/gui/entities`, `/docs`; `GET /shutdown` (not executed)	Partly documented
Body content sniffing (JSON body accepted under an XML Content-Type)	Undocumented

What we learned. Basic CRUD is solid and error messages are clear. The weaknesses are in *relationships* and *consistency*: category links are one-directional (BUG-01/02/03), deletes leave ghost links (BUG-10/11), GETs on missing parents return data (BUG-04), PUT semantics are inconsistent with the docs and destroy links (BUG-05/09), booleans are strings on output but refused as strings on input (BUG-06), failed requests burn ids (BUG-07) and XML cannot create relationships (BUG-12).

Concerns: referential integrity, no title length limit, silent data loss on PUT. **New ideas:** concurrency, very large payloads, link cycles, duplicate links, performance with many objects (Part C).

3. Structure of the unit test suite

Module (`tests/`)	Tests	Covers
test_todos_api.py	29	todo CRUD, filters, invalid ops, side effects
test_projects_api.py	20	project CRUD, errors, side effects
test_categories_api.py	19	category CRUD, errors, side effects
test_todo_relationships_api.py	15	/todos/:id/tasksof, /categories
test_project_relationships_api.py	14	/projects/:id/tasks, /categories
test_category_relationships_api.py	11	/categories/:id/todos, /projects
test_payload_formats_api.py	22	JSON/XML in & out, malformed JSON, malformed XML, empty/plain bodies
test_undocumented_api.py	19	HEAD, OPTIONS, filters, GUI/docs pages, unicode, long titles
test_interoperability_api.py	8	cross-entity workflows
test_bugs_expected_behavior.py	14	documented/expected behaviour that fails (xfail strict)
test_bugs_actual_behavior.py	12	the actual behaviour behind each bug (passes)
test_service_availability.py	2	service reachable; connection error type

Per-test lifecycle (`conftest.py`, autouse fixture): (1) *ensure ready* – snapshot all three collections; if the service is down the

fixture fails, so every test errors rather than skips; (2) *set up* – each test creates its own objects through `api_client` helpers and never depends on absolute ids; (3) *execute / assess* – assertions check application logic (stored fields, relationship lists, other objects unchanged), not only status codes; (4) *restore* – objects created by the test are deleted and any default object whose fields or links changed is put back. Tests are independent and can run in any order: `pytest --random-seed N` shuffles with `random.Random(N)` and prints the seed.

Bug pairs. For every behaviour that differs from the documentation there is one test showing the expected behaviour failing (`xfail(strict=True, raises=AssertionError)`, so it becomes a red XPASS once fixed; `--runxfail` shows raw failures) and one showing the actual behaviour working. **Clean-code choices:** small single-purpose tests with intention-revealing names, one thin client module, no commented-out code, no shared mutable state.

4. Source code repository

```
ECSE429_PartA/
  tests/            api_client.py  conftest.py  test_*.py (12 modules)
  exploratory/      common.sh  session1_todos.sh … session4_interoperability.sh
  logs/             exploratory_session*.log  run_*.log      session_notes/  CH1-CH4_*_Notes.pdf
  bug_reports.md    README.md  Part_A_Report.pdf  PartA_UnitTests_Video.mp4
```

Run: `java -jar runTodoManagerRestAPI-1.5.5.jar` then `pip install pytest requests` and `python -m pytest tests` (optionally `--random-seed 7`). Base URL override: `TODO_API_URL`.

5. Findings of unit test suite execution

Run	Command	Result
Default order	`pytest tests`	171 passed, 14 xfailed
Random order	`--random-seed 20261014`	171 passed, 14 xfailed
Random order	`--random-seed 7`	171 passed, 14 xfailed
Random order	`--random-seed 31337`	171 passed, 14 xfailed
Service stopped	`pytest tests`	185 errors (tests fail, none skipped)
Raw expected behaviour	`--runxfail -k expected`	14 failed (the 14 bug checks, 12 distinct bugs)

Suite runtime ≈ 5–8 s. Results were identical across orders, supporting independence. Ids are monotonic and cannot be restored (BUG-07 related), so restoration is by content, not by id. The `/shutdown` endpoint was intentionally not exercised because it would stop the service under test.

Bug summary (80-char executive summaries)

ID	Summary
BUG-01	Todo->category link is not visible from GET /categories/:id/todos
BUG-02	Category->todo link is not visible from GET /todos/:id/categories
BUG-03	Project<->category link is visible only from the side that created it
BUG-04	GET on relationship of a nonexistent parent returns 200 with other data
BUG-05	PUT /todos/:id requires title although docs say 'fields to amend'
BUG-06	doneStatus returned as string but string input is rejected
BUG-07	Failed (400) creations still consume object ids
BUG-08	GET /todos/:id/tasksof/:id returns 404, swagger documents 405
BUG-09	PUT /projects/:id silently removes the project's task links
BUG-10	Deleting a category leaves a ghost link listed on the todo
BUG-11	Deleting a todo leaves a ghost link listed on the category
BUG-12	Relationship creation fails when the id is sent as XML

6. Bug report form and reports

Form fields: executive summary (≤80 chars), description, potential impact, steps to reproduce. Tracked in `bug_reports.md`.

BUG-01: Todo->category link is not visible from GET /categories/:id/todos

Description	A link made with POST /todos/:id/categories is stored one way only. The category's todos list stays empty.
Impact	Clients see different data depending on which side they query; category views miss todos.
Steps	1. Create todo T and category C 2. POST /todos/T/categories {"id":"C"} -> 201 3. GET /categories/C/todos -> {"todos":[]} (expected T)

BUG-02: Category->todo link is not visible from GET /todos/:id/categories

Description	Mirror of BUG-01: POST /categories/:id/todos is not reflected on the todo.
Impact	Same inconsistency as BUG-01 in the other direction.
Steps	1. Create T and C 2. POST /categories/C/todos {"id":"T"} -> 201 3. GET /todos/T/categories -> empty (expected C)

BUG-03: Project<->category link is visible only from the side that created it

Description	Links made from either side (project or category) are stored one way only.
Impact	A category cannot list its projects, and a project cannot list its categories.
Steps	1. Create project P and category C; POST /projects/P/categories {"id":"C"} -> 201 2. GET /categories/C/projects -> empty (expected P) 3. Mirror: POST /categories/C/projects {"id":"P"} -> 201; GET /projects/P/categories -> empty

BUG-04: GET on relationship of a nonexistent parent returns 200 with other data

Description	GET /todos/987654/categories, /projects/987654/tasks and /todos/987654/tasksof return 200 and a list of related objects of OTHER parents.
Impact	Wrong data returned silently; clients cannot detect a bad id. Possible information leak.
Steps	1. Link any todo to a category 2. GET /todos/987654/categories -> 200 containing that category (expected 404)

BUG-05: PUT /todos/:id requires title although docs say 'fields to amend'

Description	Swagger describes PUT as amending with the fields in the body; a body without title is rejected with 400.
Impact	Clients following the docs cannot do partial updates with PUT.
Steps	1. Create todo 2. PUT /todos/ID {"description":"only"} -> 400 'title : field is mandatory' (expected 200)

BUG-06: doneStatus returned as string but string input is rejected

Description	Responses contain "doneStatus":"false" yet sending "false" (string) gives 400 'should be BOOLEAN'; only JSON false works.
Impact	Round-tripping a GET response into a PUT/POST fails; typing is inconsistent.
Steps	1. GET /todos/1 -> doneStatus "false" 2. PUT /todos/1 {"title":"x","doneStatus":"false"} -> 400 (expected 200)

BUG-07: Failed (400) creations still consume object ids

Description	Each rejected POST /todos advances the id counter, leaving gaps in ids.
Impact	Id exhaustion/gaps; ids unpredictable; minor information about request volume.
Steps	1. Create todo (id N) 2. Send two invalid POSTs (empty title, bad doneStatus) 3. Create todo -> id N+3 (expected N+1)

BUG-08: GET /todos/:id/tasksof/:id returns 404, swagger documents 405

Description	Relationship instance URLs accept only DELETE; GET is documented as 405 but returns 404 even when the link exists.
Impact	Wrong status code; clients cannot distinguish 'unsupported' from 'not found'.

Steps	1. Link todo T to project P 2. GET /todos/T/tasksof/P -> 404 (expected 405)

BUG-09: PUT /projects/:id silently removes the project's task links

Description	Replacing a project with PUT resets omitted fields AND drops all task relationships.
Impact	Silent data loss of relationships on an ordinary rename.
Steps	1. Create project P and link todo T 2. PUT /projects/P {"title":"renamed"} -> 200 3. GET /projects/P/tasks -> empty (T still exists)

BUG-10: Deleting a category leaves a ghost link listed on the todo

Description	After DELETE /categories/C, GET /todos/T/categories still returns C with its title; GET /categories/C is 404.
Impact	Referential integrity broken; deleted data still exposed; clients dereferencing it fail.
Steps	1. Link T to C from the todo side 2. DELETE /categories/C -> 200 3. GET /todos/T/categories -> still lists C (expected empty)

BUG-11: Deleting a todo leaves a ghost link listed on the category

Description	After DELETE /todos/T, GET /categories/C/todos still returns T with full details.
Impact	Same integrity problem as BUG-10.
Steps	1. Link C to T from the category side 2. DELETE /todos/T -> 200 3. GET /categories/C/todos -> still lists T (expected empty)

BUG-12: Relationship creation fails when the id is sent as XML

Description	POST /todos/:id/categories with <category><id>N</id></category> returns 404 'Could not find thing matching value for id'; JSON works.
Impact	XML clients cannot create relationships although XML is advertised for all payloads.
Steps	1. Create T and C 2. POST /todos/T/categories with Content-Type application/xml body <category><id>C</id></category> -> 404 (expected 201)

Appendix A – Selected evidence

A.1 Documented vs actual (swagger statement → observed)

Documented	Observed	Bug
PUT /todos/:id: “body containing the fields to amend”	400 unless title present	BUG-05
GET /todos/:id/tasksof/:id: 405	404	BUG-08
POST relationship: “relationship between X and the category in the body”	Link visible from one side only (categories)	BUG-01..03
GET relationship: 200 (parent assumed to exist)	200 + unrelated data for missing parent	BUG-04
XML supported when Content-Type is application/xml	Works for objects, 404 for relationship bodies	BUG-12
DELETE object: “Deleted a specific ...”	Object gone, links to it remain on the other side	BUG-10/11

A.2 Exploratory log excerpt (session 4, ghost link – BUG-10)

```
$ curl -X DELETE http://localhost:4567/categories/222        -> HTTP 200
$ curl -X GET    http://localhost:4567/todos/406/categories
{"categories":[{"id":"222","title":"interop category","description":""}]}   -> HTTP 200
```

A.3 Test pair excerpt (BUG-10)

```
@bug("BUG-10", "deleting a category leaves a ghost link on the todo")
def test_deleting_category_removes_it_from_todo_categories():
    todo, category = api.create_todo(), api.create_category()
    api.link(f"/todos/{todo['id']}/categories", category["id"])
    api.delete(f"/categories/{category['id']}")
    assert api.get(f"/todos/{todo['id']}/categories").json() == {"categories": []}

def test_bug10_deleted_category_still_listed_on_todo():          # actual behaviour
    ...
    assert api.ids_of(api.get(f"/todos/{todo['id']}/categories"), "categories") == [category["id"]]
    assert not api.exists("categories", category["id"])
```

A.4 Fixture excerpt (ready → snapshot → restore)

```
@pytest.fixture(autouse=True)
def system_ready_and_restored():
    try:
        before = _snapshot()
    except Exception as error:
        pytest.fail(f"Todo manager is not running at {api.BASE_URL}: {error}")
    yield before
    after = _snapshot()
    _delete_objects_created_by_test(before, after)
    ...restore default objects whose fields or links changed
```

A.5 Suite output (seed 7)

```
[random order] seed=7
...  171 passed, 14 xfailed in 5.54s
```

A.6 Test-type coverage against the project requirements

Requirement	Where
Confirm API does what it should (logic, not just codes)	entity and relationship modules re-read stored state
Documented vs actual: two modules	test_bugs_expected_behavior / test_bugs_actual_behavior
No unexpected side effects	“does not touch siblings/other objects” tests + fixture restore check
JSON and XML payloads	test_payload_formats_api
Malformed JSON / XML	test_payload_formats_api (4 tests)
Command-line queries	exploratory curl scripts; filters in test_todos/projects/categories/undocumented
Return codes	201/200/400/404/405 asserted throughout

Invalid operations (double delete etc.)	every entity and relationship module
Fail if service down	autouse fixture; logs/run_service_down.log

Appendix B – Generative AI usage (course requirement)

Purpose. Generative AI was used mainly to support the exploratory testing in Part A: planning the charter-driven sessions and the probes for each capability, suggesting probes that the team then ran against the application, and analysing and organising the observed behaviour, session findings, and bug reports.

Scope. Generative AI had a limited role in the code. It helped format and tidy code the team had already written, such as consistent layout and naming. It did not design or write the test logic: the unit test suite, the exploratory scripts, and all test cases and assertions were written by the team.

Representative prompts.

- "Help us plan a charter-driven exploratory testing session for one capability of the Todo List Manager: suggest probes that exercise documented and undocumented behaviour with typical data. Do not write any code."
- "Here are the API responses we observed during this session. Help us spot behaviour that differs from the documentation and group the findings into candidate bugs."
- "Check these bug reports against the Part A grading sheet: each one needs a summary of 80 characters or less, a description, a potential impact, and steps to reproduce. Flag anything that is missing."

Verification. All results reported here were produced by running the real application. The team independently re-ran the test suite and reviewed all findings, session notes, and bug reports before submitting.